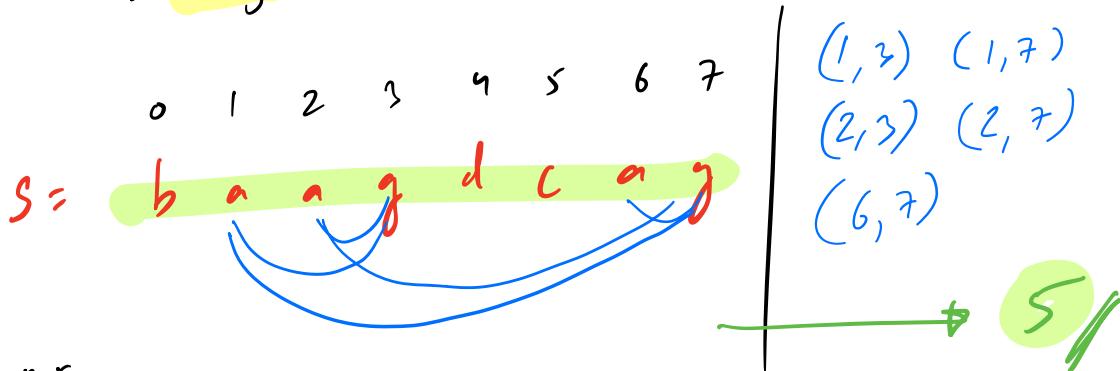


* Carry Forward

Count the pairs "ag"

Given a string, calculate the no. of pairs (i, j)
 $i < j$ & & $s[i] == 'a'$ & & $s[j] == 'g'$



I) BF

```

cnt = 0;
for (i = 0; i < N; i++) {
    for (j = i + 1; j < N; j++) {
        if (A[i] == 'a' & &
            A[j] == 'g')
            cnt++;
    }
}
return cnt;

```

TC = $O(N^2)$

SC = $O(1)$

```

cut = 0;
for (i = 0; i < N; i++) {
    if (A[i] == 'a') {
        for (j = i + 1; j < N; j++) {
            if (A[j] == 'g') {
                cut++;
            }
        }
    }
}
return cut;

```

TC = $O(N^2)$

SC = $O(1)$

Worst case: "a a a a a - - a"

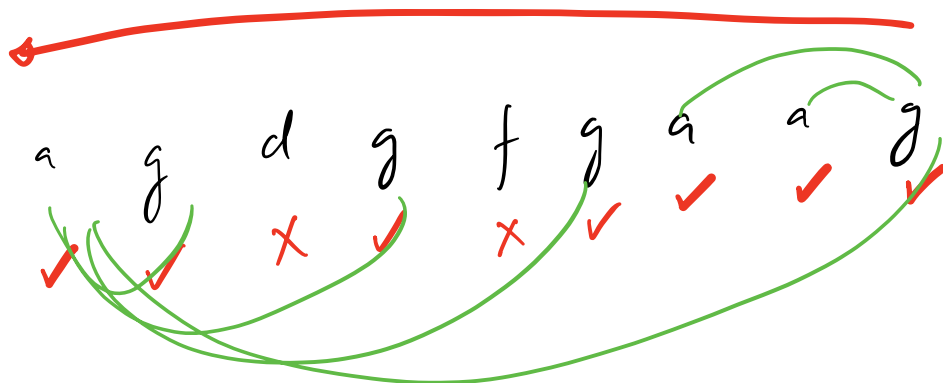
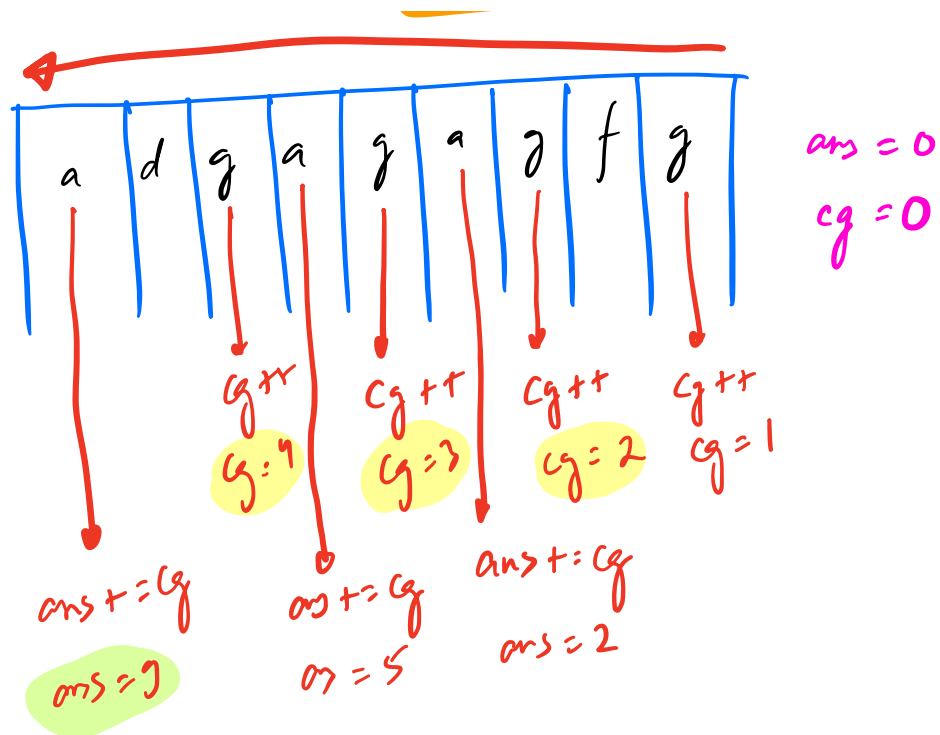
S = a d g a g a g f g

0 1 2 3 4 5 6 7 8

↑ ↑ ↑ →

4 + 3 + 2 = 9

Obs: for every 'a', count the no of 'g' on the right side of it!



$$cg = 0 \times 1 \times 2 \times 3 \times 4$$

$$ans = 0 \times 1 \times 2 \times 3 \times 4$$

cg = 0, ans = 0;

for (i = N-1; i >= 0; i--) {

if (A[i] == 'g') {

cg++;

} else if (A[i] == 'a') {

ans += cg;

}

return ans;

$N \times O(1)$

$TC = O(N)$

$SC = O(1)$

a d g a g a g f g
✓ x ✓ ✓ ✓ ✓ x ✓
cg = 0 x x 3
ans = 0 x 7 6 9

III Prefix Sum

S:	a	d	g	a	g	a	g	f	g
A:	1	0	0	1	0	1	0	0	0
PS:	1	1	1	2	2	3	3	3	3

$TC = O(N)$
 $SC = O(N)$

Q

"a x g"

(i, j, k)

$i < j < k$

$A[i] = 'a', A[j] = 'x', A[k] = 'g'$

S:	a	x	a	a	x	g	x	g	a	a	x
PA:	1	1	2	3	3	3	3	3	4	5	5
SG:	2	2	2	2	2	2	1	1	0	0	0

BF $\rightarrow N^3$

$TC = O(N)$

$SC = O(N)$

Q Given an array A.

Find the no. of leaders in the array!

An element is a leader if it is greater than all the elements on its right side!

NOTE: $A[N-1]$
is ALWAYS a leader!

A: 15 -1 7 2 5 4 2 3 → 5

✓ ✗ ✓ ✗ ✓ ✓ ✗ ✓

I) BF

cnt = 0;
for (i = 0; i < N; i++) { → N

mc = -∞;

for (j = i + 1; j < N; j++) { → ~N

mc = max(mc, A[j]); } → if (A[j] > mc)
mc = A[j]

if (A[i] > mc) {
cnt++;

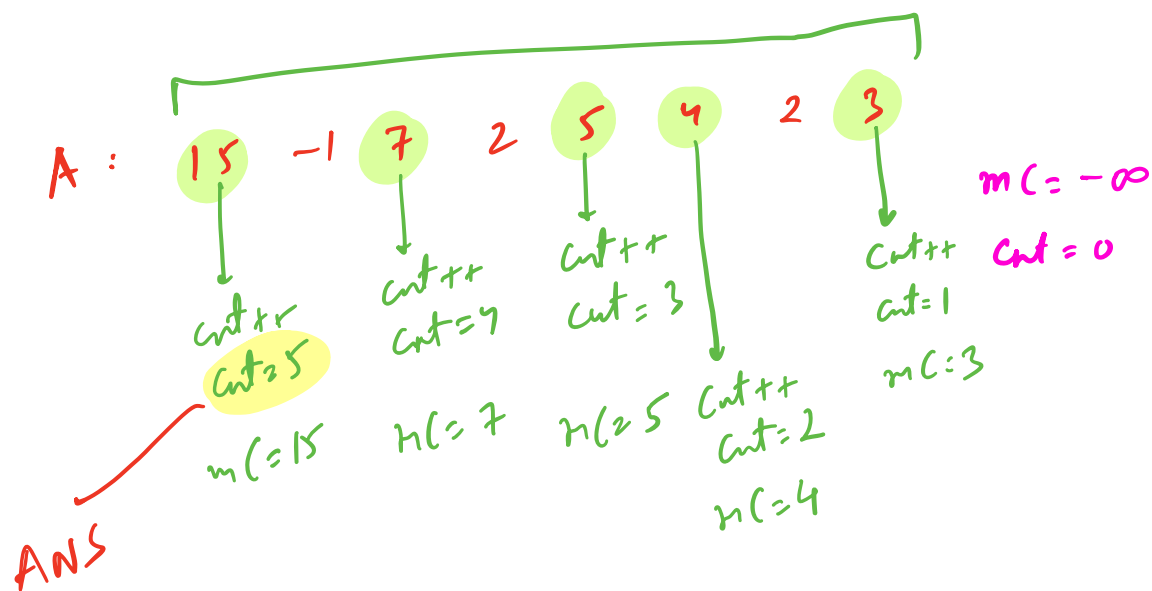
}

return cnt;

TC = $O(N^2)$

SC = $O(1)$

II



```

m = -∞, cnt = 0;
for (i = N-1; i ≥ 0; i--) {
    if (A[i] > m) {
        cnt++;
        m = A[i];
    }
}
return cnt;

```

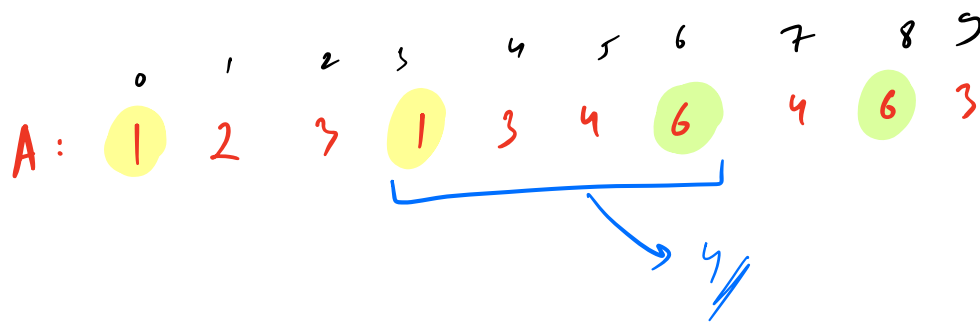
→ N × O(1)

$T.C = O(N)$

$S.C = O(1)$

Q Closest Min Max

Given an array, find the length of the smallest subarray which contains both the MIN & MAX of the array!



I) BF

1. Find MIN $\rightarrow N$
2. Find MAX $\rightarrow N$
3. $f(L=0 \rightarrow N-1) \{$

ANS = N

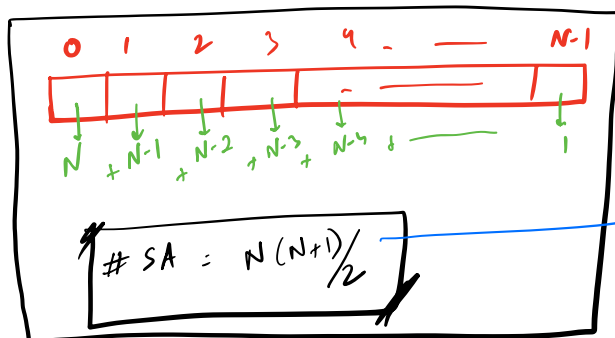
$f(R=L \rightarrow N-1) \{$
 $// [L, R]$

RUN a for loop $L \rightarrow R$ to check if it contains both MIN & MAX

true: $ANS = \min(ANS, R-L+1);$

false: $\times$

$TC = O(N^3)$
 $SL = O(1)$



$\sim N^2$

A: 8 8 8 8 $\rightarrow$ ANS = 1
 MIN $\rightarrow$ 8 | MAX $\rightarrow$ 8

if (MIN == MAX)
 ANS = 1

A: 2 2 6 4 5 1 5 2 6 4 1
 MAX MIN MAX MIN

Obs

1. - MIN - MAX - MIN - $\rightarrow$ X

MAX - MAX - MIN X

the ANS S.A. would contain
 EXACTLY 1 occ of both MIN & MAX!

2. - MIN - MAX - X
 ↓
 MIN - MAX

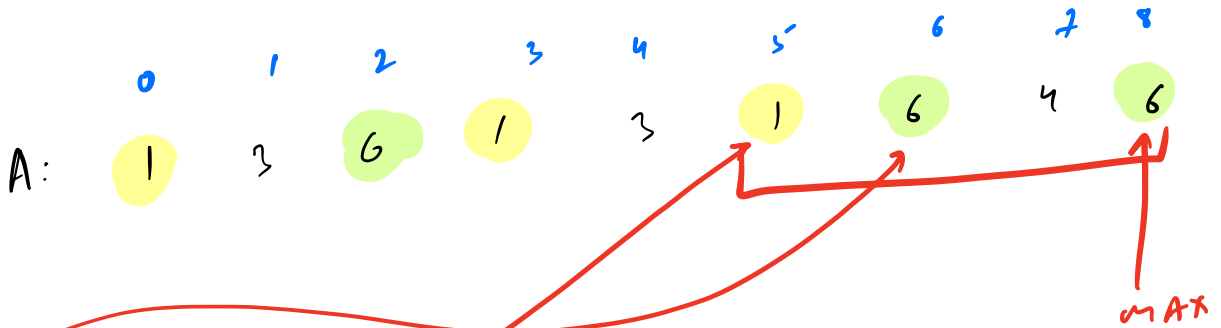
the ANS S.A. would contain
 MIN & MAX at boundaries!

→ Gr I :

MIN — MAX

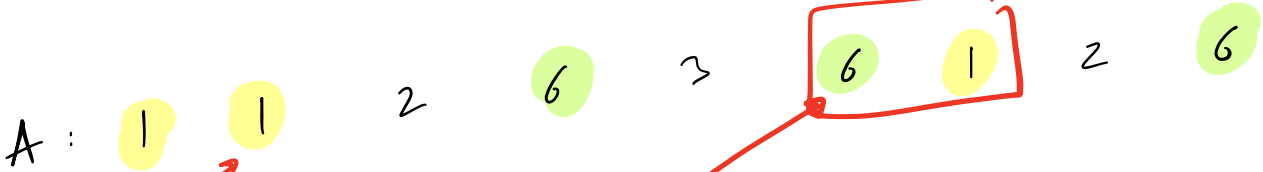
→ Gr II :

MAX — MIN



MAX_i
MIN_i

ANS = ~~7~~
2



MIN_i

MAX_i

ANS = ~~7~~
2

1. Find MIN $\rightarrow N^+$
2. Find MAX $\rightarrow N^+$ if (MIN == MAX) ret 1;

3. $MIN_i = -1$, $MAX_i = -1$; $ANS = N$;

f(i = 0 $\rightarrow$ N-1) $\rightarrow N^+$

if (A[i] == MIN) {

if (MAX_i != -1) {

$ANS = \min(ANS, i - MAX_i + 1);$

}

$MIN_i = i;$

} else if (A[i] == MAX) {

if (MIN_i != -1) {

$ANS = \min(ANS, i - MIN_i + 1);$

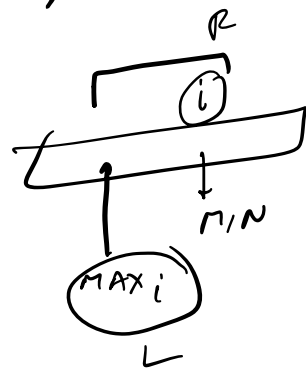
}

$MAX_i = i;$

}

}

ret ANS;



$O(1)$

$TC = O(N)$

$SC = O(1)$